

Тестовое задание

Junior QA Engineer · Manual + API Testing

Уровень:	Junior QA Engineer (QA-инженер)
Стек:	Ручное тестирование, тест-дизайн, Postman/Insomnia (API), базовые SQL-запросы
Формат сдачи:	Документ с тест-кейсами и баг-репортами + Postman-коллекция (экспорт .json) + краткое резюме

Это задание — не экзамен на знание теории тестирования. Мы смотрим на то, как ты думаешь: умеешь ли ты разбирать требования на сценарии, находить неочевидные баги и внятно их описывать так, чтобы разработчику не пришлось переспрашивать «а как воспроизвести?».

Тебе предстоит протестировать страницу онлайн-бронирования столика — реальную фичу из нашего продукта — и её API-эндпоинт. Задание построено на тех же требованиях, по которым эту фичу разрабатывают наши frontend-стажёры, так что ты будешь тестировать «настоящую» бизнес-логику, а не абстрактный пример.

** Не старайся охватить всё. Лучше меньше тест-кейсов, но глубоких и точных.*

1. Что тестируем

Ссылку на демо-версию страницы бронирования (со специально оставленными дефектами) и описание/коллекцию API-эндпоинта пришлёт рекрутер вместе с заданием. Ниже — требования к фиче, на основе которых нужно строить тест-кейсы.

Форма бронирования

- Поля: имя гостя, номер телефона, дата, время, количество гостей (1–12). Все поля обязательны.
- Время — только слоты: 12:00, 13:00 ... 22:00, шаг 1 час.
- Телефон — российский формат: +7XXXXXXXXXX или 8XXXXXXXXXX (10 цифр после кода). Допустимы пробелы, скобки и дефисы при вводе.
- Дата — не раньше сегодняшнего дня и не позднее +90 дней.
- Имя — минимум 2 символа, только буквы, пробелы, дефис.
- Ошибки валидации показываются рядом с полем (не alert), как при попытке отправки, так и при потере фокуса.

Отправка и подтверждение

- При отправке — лоадер на кнопке («Бронирую...»), кнопка disabled.
- После успешной отправки — экран подтверждения с именем, датой, временем и числом гостей.
- Кнопка «Забронировать ещё» возвращает к пустой форме.

API бронирования

- Эндпоинт принимает те же поля, что и форма, и применяет те же правила валидации на бэкенде.
- Корректный запрос — 201/200 с телом брони. Некорректный — 400 с понятным сообщением об ошибке по каждому полю.
- Описание эндпоинта и пример запроса будут в присланной Postman-коллекции.

2. Что нужно сделать

Блок	Требование
Тест-план / чек-лист	Краткий чек-лист сценариев тестирования формы бронирования (позитивные, негативные, граничные значения) — до начала тестирования.
Ручное тестирование	Пройти по чек-листу на демо-странице, зафиксировать фактическое поведение.
Тест-кейсы	Не менее 12 тест-кейсов в табличном виде: шаги, ожидаемый результат, фактический результат, статус (Pass/Fail). Обязательно покрыть валидацию телефона, даты и количества гостей.
Баг-репорты	Оформить найденные дефекты (ожидаем от 3 до 8 — часть багов оставлена намеренно): заголовок, шаги воспроизведения, ожидаемый и фактический результат, severity/priority, окружение.
API-тестирование	В Postman/Insomnia проверить минимум 5 сценариев (успешный запрос + 4 негативных: невалидный телефон, дата в прошлом, нехватка обязательного поля, некорректное количество гостей). Экспортировать коллекцию.
Отчёт о тестировании	Короткая сводка: сколько тест-кейсов, сколько багов, критичность, общий вывод по готовности фичи к релизу.

Пример строки тест-кейса

ID	Шаги	Ожидаемый результат	Статус
TC-07	Ввести телефон "89991234567", нажать "Забронировать"	Телефон принят как валидный, ошибки нет	Fail — форма показывает ошибку

3. Чего делать не нужно

- Писать автотесты (будет плюсом, но не обязательно) — приоритет на ручном тестировании и тест-дизайне.
- Тестировать нагрузку/производительность.
- Искать баги, не относящиеся к описанным выше требованиям (дизайн, орфография и т.п. — не в фокусе).

4. Оформление

- Тест-кейсы и баг-репорты — в Google Sheets, Notion или markdown-таблице (на твой выбор), ссылка должна быть открыта для просмотра.
- Postman-коллекция — файл .json, экспортированный из Postman/Insomnia.
- В сопроводительном резюме (2–3 предложения): какие решения принял при построении тест-плана и что бы протестировал ещё при наличии времени.

5. Критерии оценки

Итоговая оценка — не сумма галочек. Мы смотрим на то, насколько тщательно и системно ты подошёл к тестированию и насколько понятно твои баг-репорты читаются разработчиком.

Критерий	Вес	На что смотрим
Полнота тест-кейсов	25%	Покрытие позитивных, негативных и граничных сценариев; отсутствие дублирования
Качество баг-репортов	30%	Воспроизводимость, чёткость шагов, корректная приоритизация
API-тестирование	20%	Покрыты ключевые сценарии валидации; корректно интерпретированы коды ответа
Тест-дизайн и мышление	15%	Способность находить неочевидные edge cases, логика чек-листа
Отчёт и оформление	10%	Понятная структура, читаемость, внятный вывод о готовности фичи

Бонусные баллы

- Автоматизация хотя бы одного сценария (Playwright/Selenium) поверх ручного тестирования.
- SQL-запрос для проверки, что бронь действительно записалась в БД (если предоставлен доступ к тестовой БД).
- Предложения по улучшению тестируемости фичи (например, data-testid атрибуты).

MISE — AI-первая студия для ресторанного бизнеса • Вопросы: в чат с рекрутером